

Agenda

Builder design pattern

Inner classes in Java

Production builder pattern

Builder Design Patterns

↪ Creational: obj creation

Requirements:

- ① My class has lots of attributes
- ② Some attributes require validation, if validation fails, then we should not create object.
- ③ Some attrs are compulsory & some are optional.

class Student {

↙ String name;

↙ String email;

↙ String phone Number;

↙ int age; ← > 18

String university;

String track;

int yearOfGraduation;

Approach # 1: Getters & Setters

```
class Student {
```

```
    ...
```

```
    public void setAge (int age) {  
        if (age < 18) {  
            throw new Exception();  
        }  
        this.age = age;  
    }
```

3

Pros: ① Simple

Cons: ① 2nd req is breaking

Approach #2: Monster constructor

```
class Student {
```

```
    ...
```

```
    public Student (String name, String  
        email, String uni, String phone,  
        int age, String track, int yob) {
```

```
        if (age < 18) {  
            throw new Exception ();
```

```
        }
```

```
        this.age = age;
```

```
        ...
```

```
    }
```

```
new Student ("a@b.com", "ABC", ...)
```

Pros : ① 1st & 2nd req are met

Cons : ① Due to human error, erroneous obj can be created which will later create bugs.

② All attrs are initialized in 1 go, #3 not met.

③ Add few new attrs to Student Future. This will lead to lots of code changes in the future where existing code is getting called.

Approach # 3 : Multiple smaller constructors

```
class Student {
```

```
    :
```

```
    public Student (String name, String  
        email, String phoneNumber, int age) {
```

```
        if (age < 18) {  
            throw new Exception();
```

```
        }
```

```
        this.age = age;  
        :  
    }
```

```
    public Student (String name, String  
        email) {
```

```
    ;  
    /
```

```
}
```

3

Pros:

① All req are met.

Cons:

① Constructor explosion can happen

②

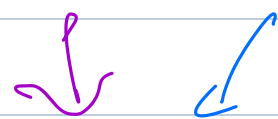
1: ^{str} name, email

2: ^{str} name, uni

Some combined constructors will be restricted due to clashing signatures.

③ New additions will lead to more constructors being created.

Approach #4: ^{Object} Map < String, ? >

public Student () {

int age = (Integer) map.get("age");

if (age < 18) {

throw new Exception();

}

this.age = age;

if (this.age != 0 &&

this.name != null ...) {

throw new Exception();

Pros :

① Meets all requirements.

Cons :

```
Map<S, O> map = new HashMap();
```

```
map.put("age", "26");
```

```
Student s = new Student(map);
```

Builder class
class Helper {

String name;

String email;

String phone Number;

int age;

String university;

String track;

int yearOfGraduation;

}

Implement basic + prod builder pattern

SOLID-1 + SOLID-2 assignments

Goal before next class:

↳ wrap up all LLD-2 assignment